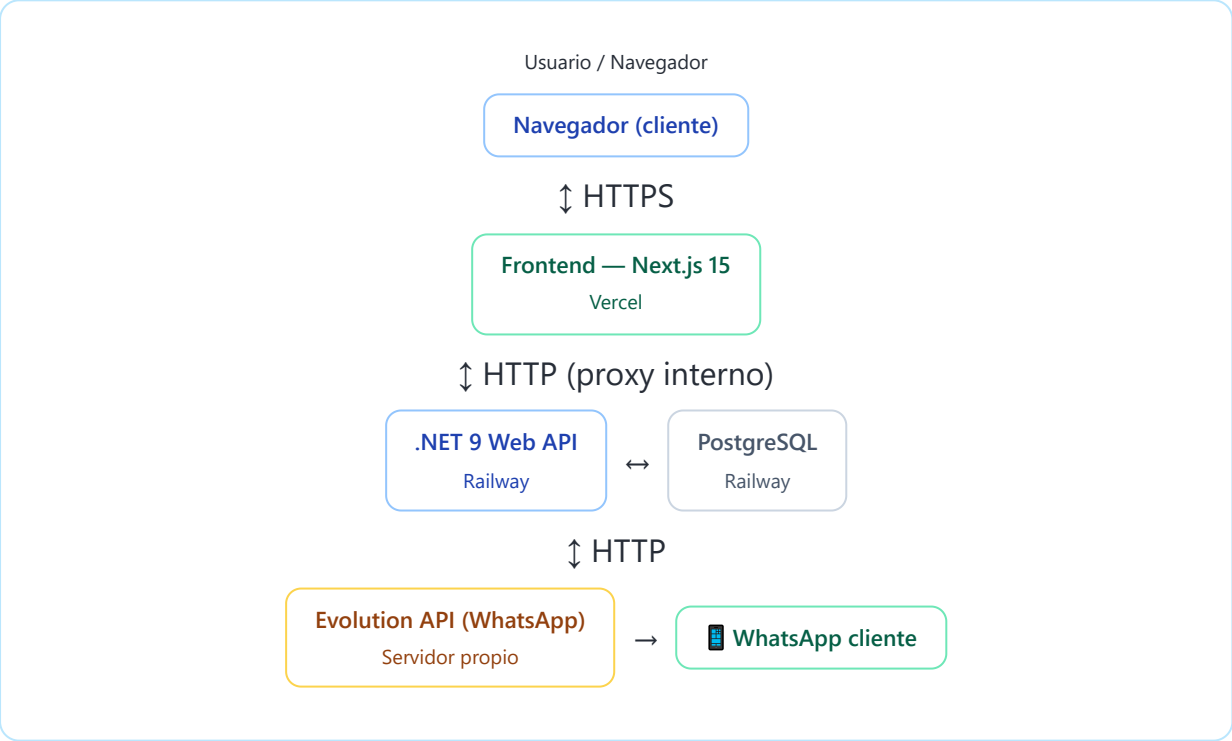
MecaFlow

Sistema de gestión para talleres mecánicos

PARTE 1 — TECNOLOGÍA Y SERVICIOS

Abril 2026

🏗️ Diagrama de Arquitectura



⚙️ Stack Tecnológico

FRONTEND

- **Next.js 15** — App Router, SSR/CSR
- **TypeScript** — tipado estático completo
- **Tailwind CSS** — utilidades de diseño
- **shadcn/ui** — componentes base (Dialog, Table, Card...)
- **Lucide React** — íconos vectoriales
- **Sonner** — notificaciones toast
- **SheetJS (xlsx)** — exportación Excel en cliente

BACKEND

- **.NET 9** — runtime y SDK
- **ASP.NET Core Web API** — controllers REST
- **Entity Framework Core 9** — ORM con migraciones
- **Npgsql** — driver PostgreSQL para EF
- **JWT Bearer Auth** — tokens firmados con HMAC-SHA256
- **BackgroundService** — health check de WhatsApp

INFRAESTRUCTURA

- **Vercel** — hosting del frontend (Next.js)
- **Railway** — hosting del backend .NET y PostgreSQL
- **GitHub** — repositorio, CI/CD automático en push
- Deploy automático en cada push a `main`
- Migraciones de BD aplicadas automáticamente al iniciar

INTEGRACIONES

- **Evolution API v1.7.4** — servidor WhatsApp propio (Baileys)
- Envío de mensajes a clientes via WhatsApp Business
- Health check automático cada 5 min con auto-reconexión
- Sesión persistida en archivos (no pierde conexión en restarts)

🔐 Autenticación y Sesión

FLUJO DE LOGIN

- Usuario envía credenciales al backend
- Backend valida y devuelve JWT firmado
- Next.js guarda el JWT en cookie `httpOnly` (no accesible desde JS)
- Cookie `mecaflow_tenant` guarda el nombre del taller (readable)
- Proxy en `/api/proxy/[...path]` inyecta el JWT en cada request

CLAIMS DEL JWT

- `tenantId` — ID del taller del usuario
- `username` — nombre de usuario
- `role` — `Admin`, `User` o `SuperAdmin`
- Issuer y Audience: `mecaflow`
- Cada query del backend filtra por `tenantId` automáticamente

🏢 Arquitectura Multi-Tenant

Cada taller (tenant) es completamente aislado. Los datos nunca se mezclan entre talleres. El `tenantId` se extrae del JWT en cada request y filtra todas las queries automáticamente.

Entidad	Relación con Tenant	Cascade delete
Usuarios	Directa (FK → Tenant)	Sí
Clientes	Directa (FK → Tenant)	Sí
Mecánicos	Directa (FK → Tenant)	Sí
Vehículos	Via Cliente → Tenant	Sí (en cascada)
Órdenes de Servicio	Via Vehículo → Cliente → Tenant	Sí (en cascada)
Items de Servicio	Via Orden → Vehículo → Tenant	Sí (en cascada)

🗄️ Modelo de Datos

Tabla	Campos principales
Tenants	Id, Name, IsActive, CreatedAt
Users	Id, TenantId, Username, PasswordHash (BCrypt), Role, CreatedAt
Customers	Id, TenantId, Name, Phone, Email, CreatedAt
Vehicles	Id, CustomerId, LicensePlate, Brand, Model, Year, Color, Notes
Mechanics	Id, TenantId, Name, Phone, Specialty, IsActive
ServiceOrders	Id, VehicleId, Status, DiagnosisNotes, InternalNotes, MileageIn, AssignedMechanic, EstimatedDeliveryAt, TotalEstimate, TotalFinal, CreatedAt, CompletedAt
ServiceItems	Id, ServiceOrderId, Description, Type (Labor/Part), Quantity, UnitPrice

🔑 Variables de Entorno

Variable	Servicio	Descripción
`ConnectionStrings__Default`	Railway (backend)	Cadena de conexión a PostgreSQL
`JWT_SECRET`	Railway (backend)	Clave secreta para firmar tokens JWT (mín. 32 chars)
`AllowedOrigins`	Railway (backend)	Origins permitidos por CORS (ej: `https://mecaflow.vercel.app`)
`Evolution__BaseUrl`	Railway (backend)	URL base de Evolution API (ej: `https://evo.midominio.com`)
`Evolution__ApiKey`	Railway (backend)	API key de Evolution API
`Evolution__Instance`	Railway (backend)	Nombre de la instancia de WhatsApp (ej: `mecaflow`)
`NEXT_PUBLIC_API_URL` / `BACKEND_URL`	Vercel (frontend)	URL del backend de Railway para el proxy

📡 Servicio de Health Check WhatsApp

WHATSAPPHEALTHSERVICE — BACKGROUNDSERVICE

- Se inicia 30 segundos después del boot de la app (para no interferir con el arranque)
- Verifica el estado de la instancia de WhatsApp cada **5 minutos**
- Si el estado es `open` → no hace nada, resetea el contador de fallos
- Si el estado es `close` → llama a `/instance/connect/{instance}` para reconectar
- Espera hasta **15 segundos** (3 intentos × 5s) verificando si alcanzó estado `open`
- Si reconecta solo → logea éxito. Si no → logea advertencia con contador de intentos fallidos
- Nunca lanza excepciones (todo el bloque está en try/catch) para no derribar la app